

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS DEPARTAMENTO ACADÉMICO DE INGENIERÍA DE SISTEMAS E INFORMÁTICA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 1

INFORME DE LABORATORIO

(formato estudiante)

INFORMACIÓN BÁSICA					
ASIGNATURA:	COMPUTACIÓN GRAFICA, VISIÓN COMPUTACIONAL Y MULTIMEDIA (E)				
TÍTULO DE LA PRÁCTICA:	<i>Procesamiento de Imágenes</i>				
NÚMERO DE PRÁCTICA:	<i>07</i>	AÑO LECTIVO:	<i>5to</i>	NRO. SEMESTRE:	<i>NOVENO</i>
FECHA DE PRESENTACIÓN	<i>dd/mm/aaaa</i>	HORA DE PRESENTACIÓN	<i>hh/mm/ss</i>		
INTEGRANTE (s) <i>Sencia Ale Bryan Daniel</i>				NOTA (0-20)	
DOCENTE(s): MBA Mg. Ing. RENE ALONSO NIETO VALENCIA.					

RESULTADOS Y PRUEBAS	
Link del video:	
Índice.	
I. Redimensionar imágenes	2
II. Crear nueva imagen con canales de color combinados	3
III. Conversión a negativo y escala de grises	4
IV. Aplicación interactiva de visualización de canales de color	4
V. Dibujo de figuras y texto en imágenes	4
VI. Aplicación de umbral binario	4
VII. Programa de dibujo interactivo con eventos de mouse y teclado	4

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS DEPARTAMENTO ACADÉMICO DE INGENIERÍA DE SISTEMAS E INFORMÁTICA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 2

I. Redimensionar imágenes

Cuando trabajamos con varias imágenes y necesitamos combinarlas o compararlas, es fundamental que tengan las mismas dimensiones. Una imagen digital es una matriz de píxeles con un alto y un ancho determinado. La función `cv2.resize()` permite cambiar esas dimensiones, escalando la imagen al nuevo tamaño usando interpolación (calcula los valores de los nuevos píxeles a partir de los originales).

Explicación.

Para poder combinar las imágenes en ejercicios posteriores, es necesario que todas tengan las mismas dimensiones. Se abrieron tres imágenes de distintos tamaños y se redimensionaron todas al tamaño de la más grande usando `cv2.resize()`.

Código usado.

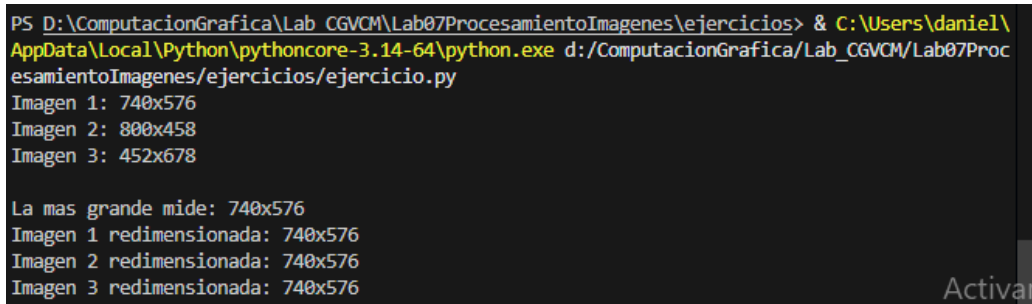
Python

```
import cv2
# cargar las 3 imagenes
rutas = ['../imagenes/img1.jpg',
         '../imagenes/img2.jpg',
         '../imagenes/img3.jpg']
imagenes = [cv2.imread(r) for r in rutas]
# tamaños originales
for i, im in enumerate(imagenes, 1):
    print(f'Imagen {i}: {im.shape[1]}x{im.shape[0]}')
# la mas grande
mayor = max(imagenes, key=lambda im: im.shape[0] * im.shape[1])
alto, ancho = mayor.shape[:2]
print(f'\nLa mas grande mide: {ancho}x{alto}')
# redimensionar
redimensionadas = [cv2.resize(im, (ancho, alto)) for im in imagenes]
# guardamos
for i, im in enumerate(redimensionadas, 1):
    cv2.imwrite(f'../resultados/1_redimensionada_{i}.png', im)
    print(f'Imagen {i} redimensionada: {im.shape[1]}x{im.shape[0]}')
print('\nListo. Revisa la carpeta resultados/')
```

Análisis del Código:

Se utiliza `cv2.imread()` para cargar cada imagen como un arreglo NumPy. La propiedad `.shape` nos da las dimensiones en formato (alto, ancho, canales). Con la función `max()` se identifica la imagen de mayor área, y con `cv2.resize()` se escalan las tres a ese tamaño.

Resultados.

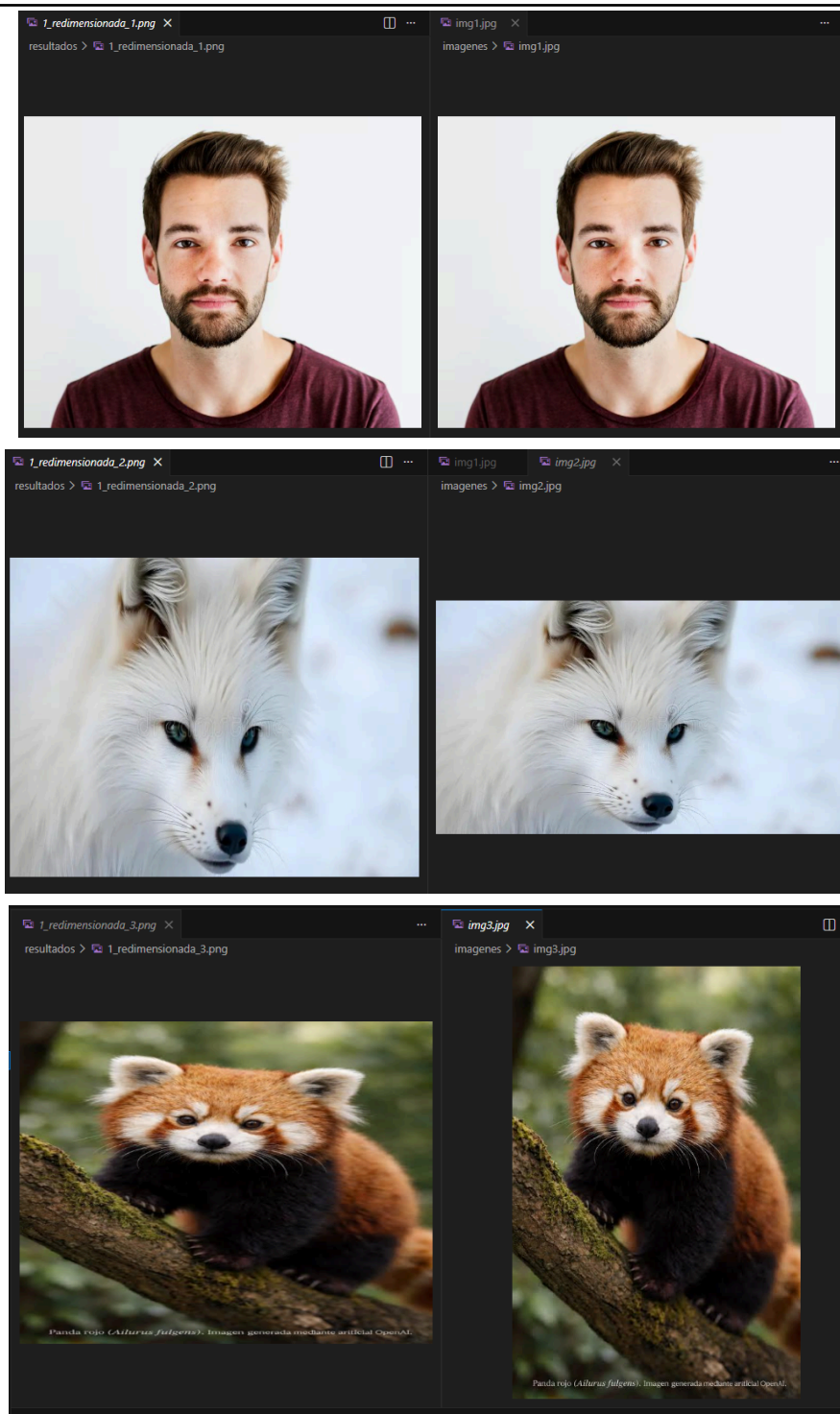


```
PS D:\ComputacionGrafica\Lab_CGVCM\Lab07ProcesamientoImagenes\ejercicios> & C:\Users\daniel\AppData\Local\Python\pythoncore-3.14-64\python.exe d:/ComputacionGrafica/Lab_CGVCM/Lab07ProcesamientoImagenes/ejercicios/ejercicio.py
Imagen 1: 740x576
Imagen 2: 800x458
Imagen 3: 452x678

La mas grande mide: 740x576
Imagen 1 redimensionada: 740x576
Imagen 2 redimensionada: 740x576
Imagen 3 redimensionada: 740x576
```

Img. 01 - Terminal mostrando los tamaños originales y los finales

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS DEPARTAMENTO ACADÉMICO DE INGENIERÍA DE SISTEMAS E INFORMÁTICA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 3



Img. 02 - Redimensionadas lado a lado

II. Crear nueva imagen con canales de color combinados

Una imagen a color en OpenCV está compuesta por tres capas llamadas canales: Azul (B), Verde (G) y Rojo (R). Cada canal es una matriz de valores entre 0 y 255 que representa la intensidad de ese color en cada

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS DEPARTAMENTO ACADÉMICO DE INGENIERÍA DE SISTEMAS E INFORMÁTICA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 4

pixel. Con la función `cv2.split()` podemos separar estos tres canales y con `cv2.merge()` podemos combinarlos libremente. En este ejercicio se crea una imagen nueva tomando el canal rojo de la primera imagen, el canal verde de la segunda y el canal azul de la tercera, produciendo un resultado con una mezcla de colores única. Es importante recordar que OpenCV almacena los canales en orden BGR (no RGB), por lo que al usar `cv2.merge()` el orden de los parámetros debe ser [B, G, R].

Código usado.

Python

```
import cv2

# cargar las 3 imagenes redimensionadas del ejercicio anterior
img1 = cv2.imread('../resultados/1_redimensionada_1.png')
img2 = cv2.imread('../resultados/1_redimensionada_2.png')
img3 = cv2.imread('../resultados/1_redimensionada_3.png')

# separar cada imagen en sus 3 canales (B, G, R)
b1, g1, r1 = cv2.split(img1) # nos interesa r1 (rojo)
b2, g2, r2 = cv2.split(img2) # nos interesa g2 (verde)
b3, g3, r3 = cv2.split(img3) # nos interesa b3 (azul)

# combinar: merge recibe en orden [B, G, R]
combinada = cv2.merge([b3, g2, r1])

cv2.imwrite('../resultados/2_combinada.png', combinada)
print(f'Imagen combinada creada: {combinada.shape[1]}x{combinada.shape[0]}')
print('Canales: Rojo=img1, Verde=img2, Azul=img3')
```

Análisis del código.

La función `cv2.split(img)` recibe una imagen a color y la descompone en tres matrices independientes, una por cada canal, devolviendo (B, G, R). Cada una de estas matrices es un arreglo bidimensional de valores entre 0 y 255.

La función `cv2.merge([b, g, r])` realiza la operación inversa: recibe una lista con tres canales y los combina en una sola imagen a color. El orden debe ser [B, G, R] porque así es como OpenCV organiza internamente los colores.

En este caso tomamos `r1` (rojo de `img1`), `g2` (verde de `img2`) y `b3` (azul de `img3`) y los unimos con `cv2.merge([b3, g2, r1])`, creando una imagen cuyos colores son una mezcla de las tres fotografías originales.

Resultados

```
PS D:\ComputacionGrafica\Lab_CGVCM\Lab07ProcesamientoImagenes\ejercicios> & C:\Users\daniel\AppData\Local\Python\pythoncore-3.14-64\python.exe d:/ComputacionGrafica/Lab_CGVCM/Lab07ProcesamientoImagenes/ejercicios/ejercicio2.py
Imagen combinada creada: 740x576
Canales: Rojo=img1, Verde=img2, Azul=img3
```

Img. 03 - Terminal mostrando la ejecución

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS DEPARTAMENTO ACADÉMICO DE INGENIERÍA DE SISTEMAS E INFORMÁTICA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 5



Img. 04 - Imagen combinada resultante, donde se aprecian los colores mezclados de las tres fotos originales

III. Conversión a negativo y escala de grises

El negativo de una imagen se obtiene invirtiendo el valor de cada pixel. Como los valores van de 0 a 255, el negativo se calcula como nuevo = 255 - original. Un pixel blanco (255) se vuelve negro (0), uno rojo intenso (255, 0, 0) se vuelve cian (0, 255, 255), y así con todos. Es el mismo efecto que el negativo de una fotografía analógica. La conversión a escala de grises transforma una imagen de 3 canales (color) a un solo canal (intensidad). OpenCV utiliza la función `cv2.cvtColor()` con la constante `COLOR_BGR2GRAY`, que aplica la fórmula ponderada $\text{Gris} = 0.299 \times R + 0.587 \times G + 0.114 \times B$. Los pesos no son iguales porque el ojo humano es más sensible al verde y menos al azul.

Sustento Matemático

Para el negativo, la operación por pixel es:

$$P'(x,y) = 255 - P(x,y)$$

Para la escala de grises, OpenCV aplica:

$$\text{Gray}(x,y) = 0.299 \times R(x,y) + 0.587 \times G(x,y) + 0.114 \times B(x,y)$$

Código usado.

Python

```
import cv2
```

```
# cargar la imagen combinada del ejercicio anterior
combinada = cv2.imread('../resultados/2_combinada.png')
```

```
# negativo: invertir cada valor de color (255 - valor)
negativo = 255 - combinada
cv2.imwrite('../resultados/3_negativo.png', negativo)
print(f'Negativo creado: {negativo.shape[1]}x{negativo.shape[0]}')
```

```
# escala de grises: convertir de BGR a un solo canal de intensidad
grises = cv2.cvtColor(combinada, cv2.COLOR_BGR2GRAY)
```


	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS DEPARTAMENTO ACADÉMICO DE INGENIERÍA DE SISTEMAS E INFORMÁTICA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 6

```
cv2.imwrite('../resultados/3_grises.png', grises)
print(f'Escala de grises creada: {grises.shape[1]}x{grises.shape[0]}')
print(f'Canales original: {combinada.shape[2]} -> Canales grises: 1')
```

Análisis del código.

La operación 255 - combinada se aplica a cada pixel de los tres canales de forma automática gracias a NumPy, que permite operaciones matemáticas sobre arreglos completos sin necesidad de recorrer pixel por pixel.

La función `cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)` convierte el espacio de color de la imagen. El parámetro `COLOR_BGR2GRAY` indica que se pasa de BGR (3 canales) a escala de grises (1 canal). La imagen resultante tiene un solo canal donde cada pixel representa la luminosidad en un valor de 0 (negro) a 255 (blanco).

Resultados.

```
PS D:\ComputacionGrafica\Lab_CGVM\Lab07ProcesamientoImagenes\ejercicios> & C:\Users\daniel\AppData\Local\Python\pythoncore-3.14-64\python.exe d:/ComputacionGrafica/Lab_CGVM/Lab07ProcesamientoImagenes/ejercicios/ejercicio3.py
Negativo creado: 740x576
Escala de grises creada: 740x576
Canales original: 3 -> Canales grises: 1
PS D:\ComputacionGrafica\Lab_CGVM\Lab07ProcesamientoImagenes\ejercicios>
```

Img. 05 - Captura de terminal mostrando la ejecución



Img. 06 - Imagen negativa: los colores aparecen invertidos respecto a la imagen combinada

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS DEPARTAMENTO ACADÉMICO DE INGENIERÍA DE SISTEMAS E INFORMÁTICA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 7



Img. 07 - Imagen en escala de grises: se observa únicamente la luminosidad sin información de color

IV. Aplicación interactiva de visualización de canales de color

Este ejercicio consiste en crear una aplicación con ventana que muestre una imagen a color y permita al usuario activar o desactivar cada canal de color (rojo, verde, azul) presionando teclas en tiempo real. Cuando un canal se desactiva, se reemplaza por una matriz de ceros (negro), eliminando ese color de la imagen. Esto permite visualizar la contribución individual de cada canal a la imagen final. La función `cv2.imshow()` crea una ventana que muestra la imagen, y `cv2.waitKey()` captura las teclas presionadas por el usuario. El programa se ejecuta en un bucle continuo que actualiza la imagen cada vez que el usuario alterna un canal.

Código usado.

```
Python
import cv2
import numpy as np

# cargar una imagen a color
img = cv2.imread('../resultados/1_redimensionada_1.png')

# separar los canales originales
canal_b, canal_g, canal_r = cv2.split(img)

# matriz de ceros del mismo tamaño (canal apagado = negro)
```

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS DEPARTAMENTO ACADÉMICO DE INGENIERÍA DE SISTEMAS E INFORMÁTICA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 8

```

ceros = np.zeros_like(canal_b)

# estado de cada canal: True = visible, False = apagado
ver_r = True
ver_g = True
ver_b = True

print('Controles: r=rojo g=verde b=azul ESC=salir')

while True:
    # si el canal esta apagado, se usa la matriz de ceros
    b = canal_b if ver_b else ceros
    g = canal_g if ver_g else ceros
    r = canal_r if ver_r else ceros

    # recomponer la imagen con los canales activos
    resultado = cv2.merge([b, g, r])
    cv2.imshow('Canales de color (r/g/b alternan, ESC sale)', resultado)

    tecla = cv2.waitKey(1) & 0xFF
    if tecla == ord('r'):
        ver_r = not ver_r
        print(f'Rojo: {"ON" if ver_r else "OFF"}')
    elif tecla == ord('g'):
        ver_g = not ver_g
        print(f'Verde: {"ON" if ver_g else "OFF"}')
    elif tecla == ord('b'):
        ver_b = not ver_b
        print(f'Azul: {"ON" if ver_b else "OFF"}')
    elif tecla == 27:
        break

cv2.destroyAllWindows()

```

Análisis del código.

La función `np.zeros_like(canal)` crea una matriz del mismo tamaño que el canal pero llena de ceros.

Cuando un canal se "apaga", se sustituye por esta matriz negra, eliminando ese color de la imagen.

La función `cv2.imshow(nombre, imagen)` abre una ventana con el nombre indicado y muestra la imagen dentro de ella. Se actualiza cada vez que se llama dentro del bucle.

La función `cv2.waitKey(1)` espera 1 milisegundo por una tecla y devuelve su código ASCII. Se usa `& 0xFF` para asegurar compatibilidad en distintos sistemas. La función `ord('r')` convierte el carácter 'r' a su código ASCII para comparar.

El operador `not` invierte el estado booleano: si `ver_r` era `True` pasa a `False` y viceversa, logrando el efecto de alternar.

Resultados.

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS DEPARTAMENTO ACADÉMICO DE INGENIERÍA DE SISTEMAS E INFORMÁTICA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 9



Img. 08 - Captura de la ventana con la imagen original, todos los canales activos



Img. 09 - Captura con el canal rojo desactivado



Img. 10 - Captura con solo el canal verde activo

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS DEPARTAMENTO ACADÉMICO DE INGENIERÍA DE SISTEMAS E INFORMÁTICA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 10

```
a:\Local\Python\pythoncore-3.14-64\python.exe d:/ComputacionGrafica/Lab CGVM/Lab07ProcesamientoIma
genes/ejercicios/ejercicio4.py
Controles: r=rojo g=verde b=azul ESC=salir
Rojo: OFF
Verde: OFF
Verde: ON
Rojo: ON
Verde: OFF
Rojo: OFF
Rojo: ON
Verde: ON
Azul: OFF
```

Img. 11 - Captura de la terminal mostrando los mensajes de ON/OFF

V. Dibujo de figuras y texto en imágenes

OpenCV permite dibujar figuras geométricas y texto directamente sobre una imagen modificando los valores de sus píxeles. En este ejercicio se utiliza un clasificador Haar Cascade, que es un algoritmo de detección de objetos entrenado para reconocer rostros frontales en imágenes. El clasificador analiza la imagen en escala de grises buscando patrones de contraste típicos de una cara (ojos, nariz, boca). Una vez detectada la posición del rostro, se dibuja un círculo sobre él con `cv2.circle()` y se añade un texto descriptivo con `cv2.putText()`. Ambas funciones modifican la imagen directamente, por lo que se trabaja sobre una copia si se quiere conservar la original.

Código usado.

```
Python
import cv2

# cargar la imagen original (usa una que tenga una cara visible)
foto = cv2.imread('../imagenes/img1.jpg')
gris = cv2.cvtColor(foto, cv2.COLOR_BGR2GRAY)
# cargar el clasificador haar cascade para rostros
clasificador = cv2.CascadeClassifier(
    cv2.data.harcascades + 'haarcascade_frontalface_default.xml')
# detectar rostros en la imagen en escala de grises
caras = clasificador.detectMultiScale(gris, scaleFactor=1.1, minNeighbors=5)
print(f'Rostros detectados: {len(caras)}')
# etiqueta segun el contenido de tu imagen
etiqueta = 'Persona' # cambia a 'Perro' o 'Gato' si es el caso

if len(caras) > 0:
    for (x, y, w, h) in caras:
        # calcular centro y radio del círculo
        centro = (x + w // 2, y + h // 2)
        radio = max(w, h) // 2
        # dibujar círculo verde de grosor 3
        cv2.circle(foto, centro, radio, (0, 255, 0), 3)
        # poner texto encima del círculo
        cv2.putText(foto, etiqueta, (x, y - 10),
                    cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 255, 0), 2)
        print(f'Cara en posicion: x={x}, y={y}, ancho={w}, alto={h}')
else:
    # si no detecta cara, dibujar al centro como respaldo
    h, w = foto.shape[:2]
```

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS DEPARTAMENTO ACADÉMICO DE INGENIERÍA DE SISTEMAS E INFORMÁTICA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 11

```
cv2.circle(foto, (w // 2, h // 2), min(w, h) // 4, (0, 255, 0), 3)
cv2.putText(foto, etiqueta, (20, 40),
            cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 255, 0), 2)
print('No se detecto rostro, se dibujo al centro')

cv2.imwrite('../resultados/5_cara_marcada.png', foto)
print('Imagen guardada con circulo y texto')
```

Análisis del código.

La función `cv2.CascadeClassifier()` carga un modelo preentrenado para detección de objetos. El archivo `haarcascade_frontalface_default.xml` viene incluido con OpenCV y está entrenado específicamente para detectar rostros humanos frontales.

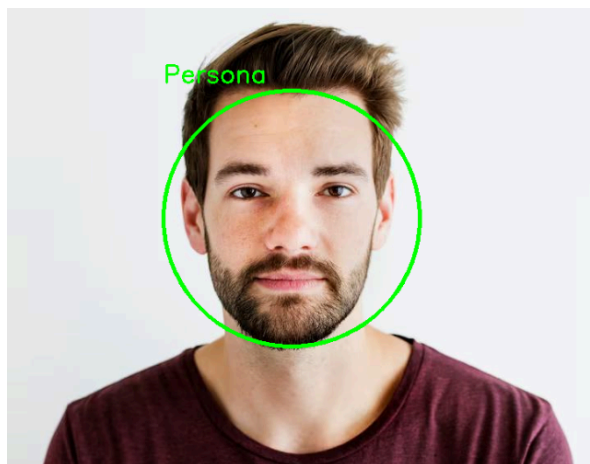
La función `detectMultiScale(imagen, scaleFactor, minNeighbors)` busca rostros en la imagen. El parámetro `scaleFactor=1.1` indica que la imagen se reduce un 10% en cada pasada de detección para encontrar caras de distintos tamaños. El parámetro `minNeighbors=5` indica cuántas detecciones vecinas se necesitan para confirmar que hay un rostro, evitando falsos positivos. Devuelve una lista de rectángulos (x, y, ancho, alto) donde encontró caras.

La función `cv2.circle(img, centro, radio, color, grosor)` dibuja un círculo. El centro es una tupla (x, y), el color se define en BGR (en este caso (0, 255, 0) es verde) y el grosor indica el ancho de la línea en píxeles. La función `cv2.putText(img, texto, posicion, fuente, escala, color, grosor)` escribe texto sobre la imagen. `FONT_HERSHEY_SIMPLEX` es una fuente sans-serif básica y la escala 1 indica tamaño normal.

Resultados.

```
PS D:\ComputacionGrafica\Lab_CGVCM\Lab07ProcesamientoImagenes\ejercicios> & C:\Users\daniel\AppData
a\Local\Python\pythoncore-3.14-64\python.exe d:/ComputacionGrafica/Lab_CGVCM/Lab07ProcesamientoIma
genes/ejercicios/ejercicio5.py
Rostros detectados: 1
Cara en posicion: x=198, y=110, ancho=315, alto=315
Imagen guardada con circulo y texto
PS D:\ComputacionGrafica\Lab_CGVCM\Lab07ProcesamientoImagenes\ejercicios>
```

Img. 12 - Captura de terminal mostrando cuántos rostros se detectaron y sus posiciones



Img. 13 - Imagen resultante con el círculo verde rodeando la cara y el texto "Persona" encima

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS DEPARTAMENTO ACADÉMICO DE INGENIERÍA DE SISTEMAS E INFORMÁTICA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 12

VI. Aplicación de umbral binario

El umbral binario (binary threshold) es una técnica de segmentación que convierte una imagen en escala de grises a una imagen de solo dos colores: blanco y negro. Se define un valor de corte (umbral), y cada pixel se compara contra él: si su intensidad es mayor al umbral se convierte en blanco (255), si es menor se convierte en negro (0). Esto permite separar objetos del fondo y simplificar la imagen para análisis posteriores como detección de contornos o reconocimiento de texto.

Sustento Matemático

La operación por pixel es:

Si $P(x,y) > \text{umbral} \rightarrow P'(x,y) = 255$ (blanco)

Si $P(x,y) \leq \text{umbral} \rightarrow P'(x,y) = 0$ (negro)

Código usado.

```
Python
import cv2

# cargar una imagen
img = cv2.imread('../imagenes/img2.jpg')
print(f'Imagen original: {img.shape[1]}x{img.shape[0]}, canales: {img.shape[2]}')

# convertir a escala de grises primero
gris = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
print(f'Imagen en grises: {gris.shape[1]}x{gris.shape[0]}, canales: 1')

# aplicar umbral binario con valor de corte 127
valor_umbral, binaria = cv2.threshold(gris, 127, 255, cv2.THRESH_BINARY)
print(f'Umbral aplicado con valor de corte: {valor_umbral}')

# contar pixeles blancos y negros para ver el resultado
blancos = (binaria == 255).sum()
negros = (binaria == 0).sum()
total = binaria.size
print(f'Pixeles blancos: {blancos} ({blancos*100//total}%)')
print(f'Pixeles negros: {negros} ({negros*100//total}%)')

cv2.imwrite('../resultados/6_grises.png', gris)
cv2.imwrite('../resultados/6_umbral_binario.png', binaria)
print('Imagenes guardadas')
```

Análisis del código.

La función `cv2.threshold(imagen, umbral, valor_maximo, tipo)` aplica el umbral a cada pixel de la imagen. Recibe la imagen en escala de grises, el valor de corte (127 en este caso, que es el punto medio entre 0 y 255), el valor máximo a asignar (255 para blanco) y el tipo de umbral. `cv2.THRESH_BINARY` indica que los pixeles por encima del umbral se ponen en blanco y los demás en negro. La función devuelve dos valores: el umbral utilizado y la imagen resultante.

Se eligió 127 como valor de corte por ser el punto medio del rango 0-255, dividiendo la imagen en dos mitades iguales de intensidad. Un umbral más bajo produce más pixeles blancos (imagen más clara), mientras que uno más alto produce más pixeles negros (imagen más oscura).

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS DEPARTAMENTO ACADÉMICO DE INGENIERÍA DE SISTEMAS E INFORMÁTICA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 13

El conteo de píxeles blancos y negros permite verificar cuantitativamente el efecto del umbral sobre la imagen

Resultados.

```
PS D:\ComputacionGrafica\Lab_CGVCM\Lab07ProcesamientoImagenes\ejercicios> & C:\Users\daniel\AppData\Local\Python\pythoncore-3.14-64\python.exe d:/ComputacionGrafica/Lab_CGVCM/Lab07ProcesamientoImagenes/ejercicios/ejercicio6.py
Imagen original: 800x458, canales: 3
Imagen en grises: 800x458, canales: 1
Umbral aplicado con valor de corte: 127.0
Píxeles blancos: 347659 (94%)
Píxeles negros: 18741 (5%)
Imagenes guardadas
PS D:\ComputacionGrafica\Lab_CGVCM\Lab07ProcesamientoImagenes\ejercicios> |
```

Img. 14 - Captura de terminal mostrando las dimensiones, el umbral y el porcentaje de píxeles blancos/negros



Img. 15 - Imagen en escala de grises usada como entrada



Img. 16 - Imagen binarizada: se distinguen claramente las zonas claras (blanco) y oscuras (negro) de la foto original

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS DEPARTAMENTO ACADÉMICO DE INGENIERÍA DE SISTEMAS E INFORMÁTICA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 14

VII. Programa de dibujo interactivo con eventos de mouse y teclado

Este ejercicio implementa un programa de dibujo que combina eventos de mouse y teclado para crear figuras geométricas sobre un lienzo en blanco. El usuario selecciona el tipo de figura con el teclado, dibuja haciendo click y arrastrando con el mouse, puede deshacer los cambios realizados y guardar el resultado final. Para el sistema de deshacer se utiliza una pila (lista en Python): antes de cada trazo se guarda una copia del estado actual del lienzo. Al presionar la tecla de deshacer, se restaura el último estado guardado con pop(), eliminando el último dibujo realizado. OpenCV permite capturar eventos del mouse mediante una función callback que se registra con cv2.setMouseCallback(). Esta función se ejecuta automáticamente cada vez que ocurre un evento como click, arrastre o soltar el botón.

Código usado.

Python

```
import cv2
import numpy as np

# lienzo blanco de 600x800 pixeles
lienzo = np.ones((600, 800, 3), dtype=np.uint8) * 255
historial = [] # pila de estados para deshacer
figura = 'linea' # figura activa por defecto
color = (0, 0, 255) # rojo en BGR
dibujando = False
ix, iy = -1, -1 # punto inicial del trazo

def dibujar(evento, x, y, flags, param):
    """callback que responde a eventos del mouse"""
    global ix, iy, dibujando, lienzo

    if evento == cv2.EVENT_LBUTTONDOWN:
        # al hacer click, guardar estado actual y punto inicial
        dibujando = True
        ix, iy = x, y
        historial.append(lienzo.copy())

    elif evento == cv2.EVENT_LBUTTONUP:
        # al soltar, dibujar la figura entre punto inicial y final
        dibujando = False
        if figura == 'linea':
            cv2.line(lienzo, (ix, iy), (x, y), color, 2)
        elif figura == 'rectangulo':
            cv2.rectangle(lienzo, (ix, iy), (x, y), color, 2)
        elif figura == 'circulo':
            radio = int(np.hypot(x - ix, y - iy))
            cv2.circle(lienzo, (ix, iy), radio, color, 2)

# crear ventana y registrar el callback del mouse
cv2.namedWindow('Dibujo interactivo')
cv2.setMouseCallback('Dibujo interactivo', dibujar)

print('Figuras: l=linea r=rectangulo c=circulo')
```

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS DEPARTAMENTO ACADÉMICO DE INGENIERÍA DE SISTEMAS E INFORMÁTICA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 15

```

print('Acciones: z=deshacer s=guardar ESC=salir')

while True:
    cv2.imshow('Dibujo interactivo', lienzo)
    tecla = cv2.waitKey(1) & 0xFF

    if tecla == ord('l'):
        figura = 'linea'
        print('Figura: linea')
    elif tecla == ord('r'):
        figura = 'rectangulo'
        print('Figura: rectangulo')
    elif tecla == ord('c'):
        figura = 'circulo'
        print('Figura: circulo')
    elif tecla == ord('z'):
        if historial:
            lienzo = historial.pop()
            print('Deshacer')
        else:
            print('No hay mas cambios para deshacer')
    elif tecla == ord('s'):
        cv2.imwrite('../resultados/7_dibujo_final.png', lienzo)
        print('Guardado como 7_dibujo_final.png')
    elif tecla == 27:
        break

cv2.destroyAllWindows()

```

Análisis del código.

La función `np.ones((600, 800, 3), dtype=np.uint8) * 255` crea una matriz de 600 filas por 800 columnas con 3 canales, llena de unos, y la multiplica por 255 para obtener un lienzo completamente blanco.

La función `cv2.setMouseCallback(ventana, funcion)` registra una función que se ejecuta automáticamente cuando ocurre cualquier evento del mouse sobre la ventana indicada. La función recibe los parámetros evento (tipo de evento), x, y (posición del cursor), flags y param.

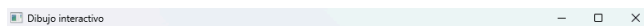
Los eventos `cv2.EVENT_LBUTTONDOWN` y `cv2.EVENT_LBUTTONUP` representan presionar y soltar el botón izquierdo del mouse respectivamente. Al presionar se registra el punto inicial, al soltar se dibuja la figura entre el punto inicial y el punto final.

La función `cv2.line(img, punto1, punto2, color, grosor)` dibuja una línea recta entre dos puntos. La función `cv2.rectangle(img, esquina1, esquina2, color, grosor)` dibuja un rectángulo definido por sus esquinas opuestas. La función `cv2.circle(img, centro, radio, color, grosor)` dibuja un círculo; el radio se calcula con `np.hypot(x-ix, y-iy)` que es la distancia euclidiana entre los dos puntos.

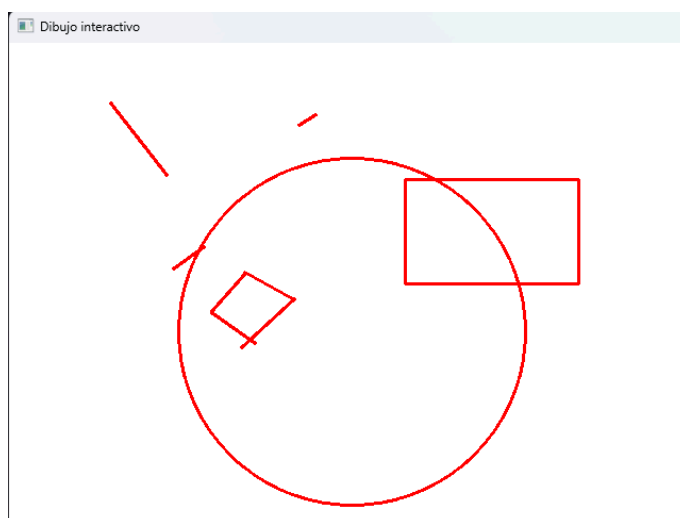
El método `lienzo.copy()` crea una copia independiente del lienzo antes de cada trazo. El método `historial.pop()` extrae y devuelve el último elemento de la lista, restaurando el estado anterior del lienzo. Este mecanismo de pila permite deshacer múltiples cambios en orden inverso.

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS DEPARTAMENTO ACADÉMICO DE INGENIERÍA DE SISTEMAS E INFORMÁTICA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 16

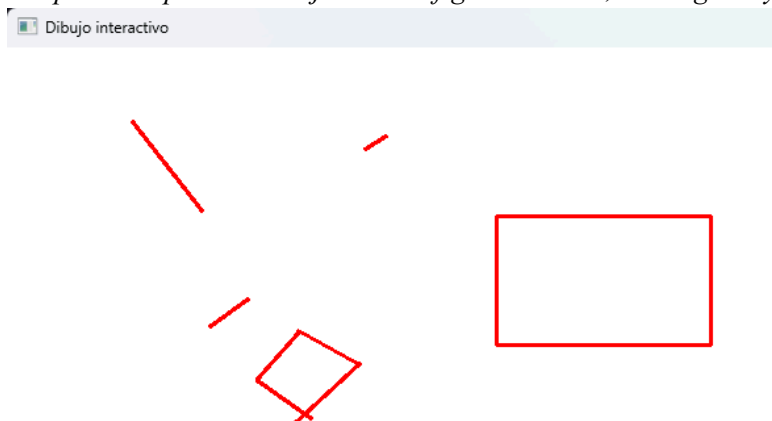
Resultados.



Img. 17 - Captura de la ventana con el lienzo en blanco al iniciar



Img. 18 - Captura después de dibujar varias figuras: líneas, rectángulos y círculos



Img. 19 - Captura después de usar deshacer, mostrando que la última figura desapareció

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS DEPARTAMENTO ACADÉMICO DE INGENIERÍA DE SISTEMAS E INFORMÁTICA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 17

```
PS D:\ComputacionGrafica\Lab_CGVM\Lab07ProcesamientoImagenes\ejercicios> & C:\Users\daniel\AppData\Local\Python\pythoncore-3.14-64\python.exe d:/ComputacionGrafica/Lab_CGVM/Lab07ProcesamientoImagenes/ejercicios/ejercicio7.py
Figuras: l=linea r=rectangulo c=circulo
Acciones: z=deshacer s=guardar ESC=salir
Figura: rectangulo
Figura: circulo
Deshacer
```

Img. 20 - Captura de la terminal mostrando los mensajes de cambio de figura, deshacer y guardar

CUESTIONARIO

1. ¿Qué otros modelos de colores existen para las imágenes?

Además del modelo RGB, que en OpenCV se representa comúnmente como BGR, existen otros modelos de color ampliamente utilizados según la aplicación. Por ejemplo, los modelos HSV (Tono, Saturación y Valor) y HSL permiten separar el color de la intensidad y el brillo, lo que facilita la segmentación de objetos por color incluso cuando existen variaciones de iluminación o sombras. Por otro lado, el modelo CMYK (Cian, Magenta, Amarillo y Negro) es el estándar empleado en procesos de impresión. También se encuentra el modelo YCrCb, que separa la información de luminancia de la información de color y es ampliamente utilizado en la compresión de imágenes y videos, como en los formatos JPEG y MPEG. Asimismo, el modelo CIELab está diseñado para aproximarse a la percepción visual humana, por lo que resulta muy útil para medir diferencias de color con mayor precisión. Finalmente, la escala de grises constituye el modelo más simple, ya que representa la imagen mediante un único canal de intensidad.

2. ¿Qué otros cambios se pueden realizar a las imágenes con OpenCV?

OpenCV ofrece una gran variedad de herramientas para el procesamiento y la manipulación de imágenes, más allá de los cambios de color. Entre sus funciones más utilizadas se encuentran los filtros de suavizado y desenfoque, como GaussianBlur y medianBlur, que ayudan a reducir el ruido presente en las imágenes. También permite detectar bordes mediante algoritmos como Canny o Sobel, así como realizar operaciones morfológicas, entre ellas la erosión y la dilatación, que sirven para eliminar imperfecciones o resaltar determinadas formas. Además, OpenCV facilita la aplicación de transformaciones geométricas, como rotaciones, escalados, volteos y cambios de perspectiva. Otras funcionalidades importantes incluyen la ecualización de histogramas para mejorar el contraste, la detección de contornos y figuras, el ajuste de brillo y contraste, el reconocimiento de objetos y rostros, y la combinación o mezcla de varias imágenes. Gracias a estas capacidades, OpenCV se ha convertido en una de las bibliotecas más utilizadas en visión por computadora.

3. ¿Cuál sería el uso del binary threshold en el procesamiento de imágenes?

El umbral binario (binary threshold) es una técnica fundamental en el procesamiento de imágenes, ya que permite segmentar una imagen y separar los elementos de interés del fondo. Su funcionamiento consiste en convertir los píxeles a únicamente dos valores, generalmente blanco y negro, según un valor de corte previamente definido. De esta manera, se simplifica considerablemente la información visual y se destacan únicamente las regiones relevantes para el

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS DEPARTAMENTO ACADÉMICO DE INGENIERÍA DE SISTEMAS E INFORMÁTICA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 18

análisis. Esta técnica se utiliza frecuentemente como etapa de preprocesamiento en sistemas de reconocimiento óptico de caracteres (OCR), donde es necesario separar el texto del fondo. También es muy útil en la digitalización de documentos escaneados, en la detección de contornos y en aplicaciones donde se requiere identificar, aislar o contar objetos según su intensidad. Aunque se trata de un método sencillo, constituye una de las bases más importantes en numerosos sistemas de visión por computadora.

CONCLUSIONES

El uso de cv2.split() y cv2.merge() permitió comprender que una imagen a color no es más que tres matrices de intensidad apiladas en orden BGR. Al separar los canales de tres imágenes distintas y recombinarlos se pudo observar cómo cada canal aporta información independiente al color final, lo que resulta fundamental para tareas de segmentación y análisis de color en visión computacional.

La aplicación de operaciones matemáticas simples como la inversión de valores (255 - pixel) y la comparación contra un umbral demostró que el procesamiento de imágenes se basa en manipular numéricamente los píxeles. Estas operaciones, aunque sencillas, son la base de técnicas más complejas como la detección de bordes, el reconocimiento de objetos y el preprocesamiento para OCR.

La implementación de los programas interactivos con cv2.imshow(), cv2.waitKey() y cv2.setMouseCallback() evidenció que OpenCV no solo sirve para procesar imágenes de forma estática, sino que permite crear aplicaciones en tiempo real donde el usuario interactúa directamente con la imagen mediante eventos de teclado y mouse, abriendo posibilidades para herramientas de edición y anotación visual.

METODOLOGÍA DE TRABAJO

REFERENCIAS Y BIBLIOGRAFÍA

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS DEPARTAMENTO ACADÉMICO DE INGENIERÍA DE SISTEMAS E INFORMÁTICA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 19

[1] Gordon, V. S., & Clevenger, J. L. (2020). *Computer Graphics Programming in OpenGL with C++*.
[2] D. P. Kothari, G. Awari, D. Shrimankar, A. Bhende (2019), *Mathematics for Computer Graphics and Game Programming*.